

AUTONOMOUS DRONE MAPPER SIMULATOR

High Level Design (HLD) Document

Course: Advanced Topics in Programming (0368-3015), Semester 2026B

Institution: Tel Aviv University (TAU)

Assignment: Project Assignment 1 (v2)

Document Version: 1.0.0

Date: May 2026

1. Introduction & Overview

This High-Level Design (HLD) document describes the software architecture for an autonomous 3D Drone Mapper Simulator. The system is engineered to simulate a spherical drone mapping unknown, bounded indoor environments using simulated rangefinders and positioning instruments. The program serves as a software-only verification loop, driving drone navigation autonomously via deterministic spatial graph exploration.

A core architectural requirement is the encapsulation of dimensions using explicit strong types through the `mp-units` compile-time library. This ensures strict physical unit safety across all component APIs (meters, centimeters, and degrees), fundamentally preventing structural bugs due to standard unit mismatches.

2. UML Class Diagram

The system separates simulated hardware capabilities (Mocks) from core autonomous execution layers (Algorithms and Map tracking) using object-oriented interfaces. This allows seamless transitions to rigid specifications in subsequent system variants.



Figure 2.1: Conceptual System Architecture and Interface Graph

Component Details & Responsibilities

- **SimulatorEngine:** Acts as the top-level orchestrator. It parses the initialization text configurations, instantiates components, runs the primary execution loop, outputs the resulting mapped representation, and grades the operations.
- **BuildingMap:** Formulates a sparse 3D spatial matrix leveraging hashed coordinates as unique indices. It tracks state tags: 0 (Empty), 1 (Occupied), -1 (Unmapped/Blocked), and -2 (Outside Defined Bounds).
- **Mock Hardware Stack:** MockPositionSensor directly reflects the shared coordinates mutated by MockMovementDriver, while MockLidarSensor performs line-of-sight raycasts against a private instance representing real environment layouts.

3. UML Sequence Diagram

The following sequence diagram outlines the primary simulation flow execution trace loop during a typical operational sequence.

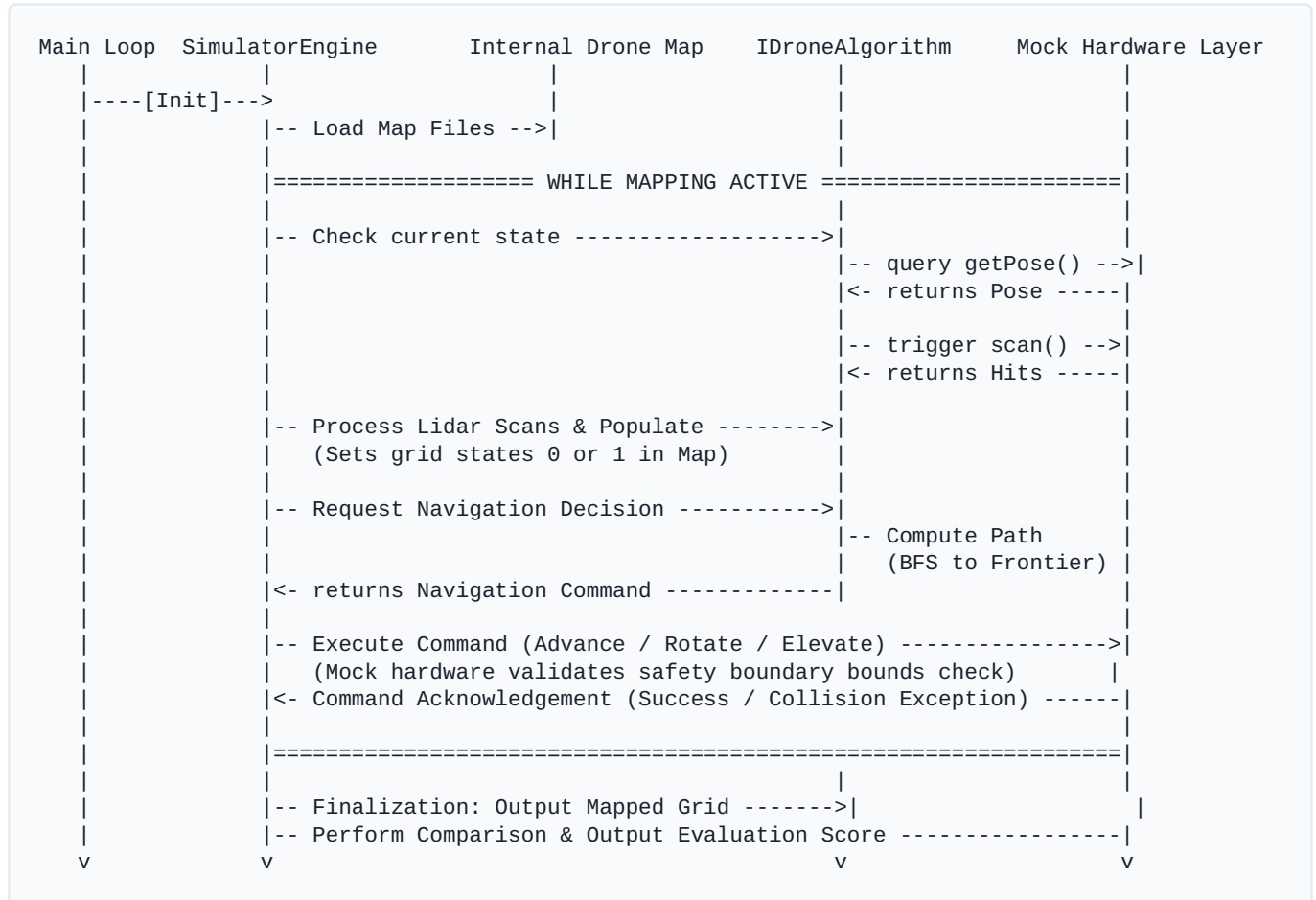


Figure 3.1: Sequential Trace of the Simulator Main Loop Execution

4. Design Considerations & Alternatives

4.1 Matrix Representation Structure

Alternative Considered: A multi-dimensional contiguous array matrix vector `std::vector<std::vector<std::vector<int>>>`. While offering highly performant constant lookup rates, structural memory scaling properties are incredibly suboptimal when processing multi-meter domains at high resolutions. For instance, a 10m x 10m x 4m volume evaluated at 1cm accuracy needs 4×10^7 indices, consuming vast memory spaces regardless of structural density.

Selected Approach: A coordinate-hashed sparse matrix structure `map std::unordered_map<CellKey, int>` using a custom spatial hashing functor. This variant scales linearly with physical objects detected inside the bounds ($O(N)$ where N represents mapped units), ensuring light operational bounds while complying cleanly with the open API criteria.

4.2 Mapping Scoring Evaluation Algorithm

To rate execution outputs cleanly, the scoring engine establishes a definitive mathematical calculation mapping real geometric environments to discovered drone charts:

$$\text{Score} = ((\text{Cells}_{\text{Correctly_Identified}}) / (\text{Total_Accessible_Valid_Cells})) \times 100$$

Accessible points are computed strictly via flood-fill tests originating at the start index. The drone configuration defines minimum spatial thresholds to determine structural passability. Narrow pathways under this limit are ignored during scoring if they prevent physical access, isolating performance ratings to attainable spaces.

4.3 Autonomous Path Exploration Mechanism

Selected Approach: A deterministic Breadth-First Search (BFS) algorithm to locate frontiers (the closest boundaries separating known free space from unknown unmapped cells). By establishing paths via a FIFO queue, execution choices remain entirely reproducible and free of random seeds. The drone checks its volume requirements against known points before choosing moves, guaranteeing safety and zero collision risk.

5. Testing Strategy

To meet the required high-reliability standards and satisfy bonus considerations, the validation framework uses a tiered testing strategy:

- **Unit Level Testing (Google Test Suite):** Validates fundamental components in isolation. Test blocks confirm that geometry utilities correctly translate vector fields via strong physical units. They also ensure spatial coordinates match when shifted by varying yaw values.
- **Geometric Lidar Verification:** Confirms raycast behavior matches the physical specification. Tests verify that objects within the Z_{min} threshold return zero distance, while elements past Z_{max} are correctly ignored.
- **Boundary & Error Exception Asserts:** Validates input error recovery. If values are missing in `mission_config.txt`, the engine falls back to default values and logs them to `input_errors.txt`. Unrecoverable issues safely exit through `main` without crashes.
- **Integration Exploration Scenarios:** Evaluates the exploration algorithm using small, enclosed test maps. These scenarios verify that the drone successfully maps accessible spaces and completes its loop when no new frontiers remain.